NAME:RAGUL.M

REG NO:192524066

SUBJECT:PYTHON PROGRAMMING

CODE:CSA0802

CO:3 AT:1

GITHUB LINK: https://github.com/192524066simats-netizen/CSA0802/tree/main/ASSISGNMENT%20TOOL/CO1

Q1 – Lists and Slicing: Daily Rainfall Records

Aim:

To write a Python program using lists and slicing to calculate weekly rainfall, maximum and minimum rainfall, moving average and dry days.

Algorithm:

1. Start the program.
2. Create a list of rainfall readings.
3. Calculate weekly totals using slicing.
4. Find maximum and minimum rainfall.
5. Calculate the 7-day moving average.
6. Identify dry days where rainfall is zero.
7. Display the results.
8. Stop the program.

Pseudocode:

START

Create rainfall list

FOR every 7 days

Calculate weekly total

END FOR

Find maximum and minimum

Calculate 7-day moving average

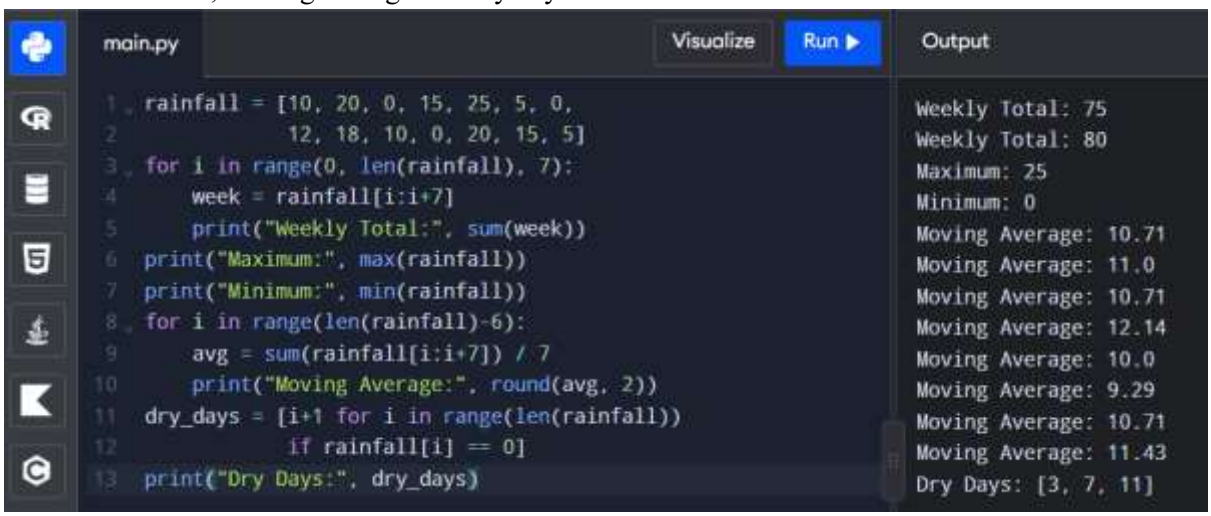
Find dry days

Display results

STOP

Result:

Thus, the Python program was executed successfully to calculate weekly rainfall, maximum and minimum rainfall, moving average and dry days.



Q2 – Tuples, Mutability and Copying

Aim:

To write a Python program using tuples and lists to demonstrate mutability, aliasing, shallow copy, deep copy and tuples as dictionary keys.

Algorithm:

1. Start the program.
2. Create a 2D list using list comprehension.
3. Demonstrate the aliasing problem.
4. Create a corrected grid.
5. Perform shallow copy and deep copy.
6. Create tuples for bus stops.
7. Use tuples as dictionary keys.
8. Display the results.
9. Stop the program.

Pseudocode:

START

Create a 2D grid

Create an aliased grid

Modify one element

Create a corrected grid

Perform shallow copy

Perform deep copy

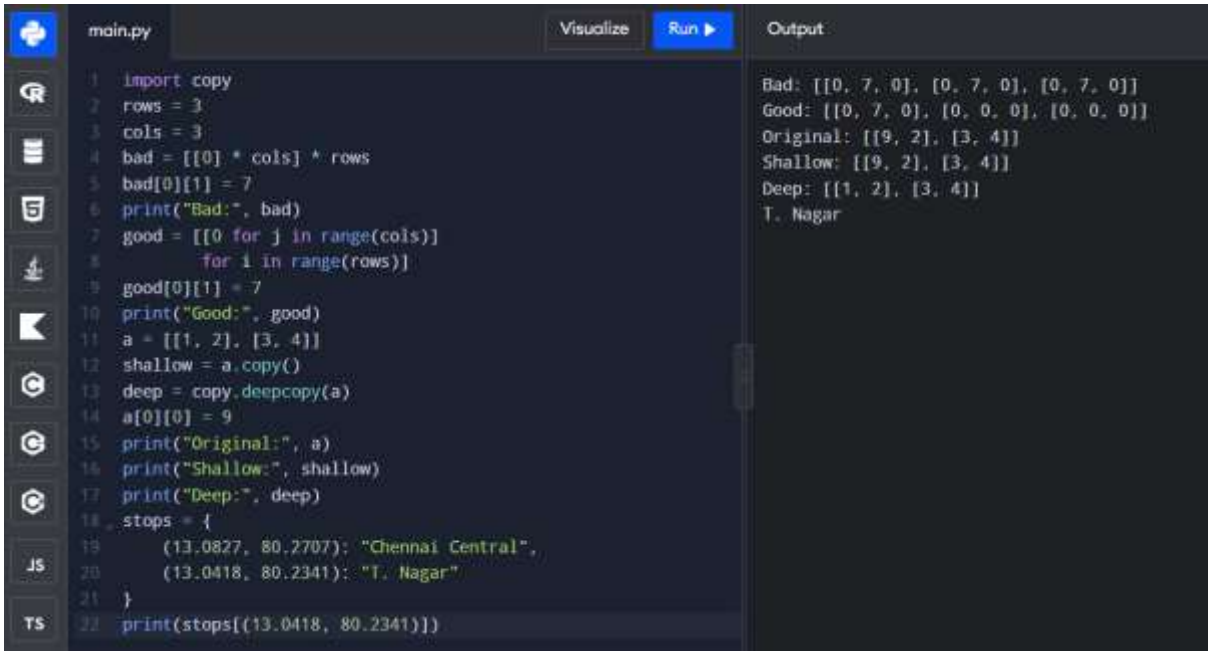
Create tuple dictionary keys

Display results

STOP

Result:

Thus, the Python program was executed successfully to demonstrate tuples, mutability, aliasing, shallow copy and deep copy.



Q3 – Dictionaries: Farmer Helpline Complaint Analysis

Aim:

To write a Python program using dictionaries to count complaint word frequencies, group complaints by district and find the top frequent words.

Algorithm:

1. Start the program.
2. Create a list of complaints.
3. Create a set of stop words.
4. Split complaints into words.
5. Count word frequencies using a dictionary.
6. Group complaints by district.

7. Sort words according to frequency.
8. Display the top three words.
9. Stop the program.

Pseudocode:

START

Create complaint list

Create stop words

Create empty dictionary

FOR each complaint

Split into words

FOR each word

IF word is not a stop word

Count word frequency

END IF

END FOR

END FOR

Group complaints by district

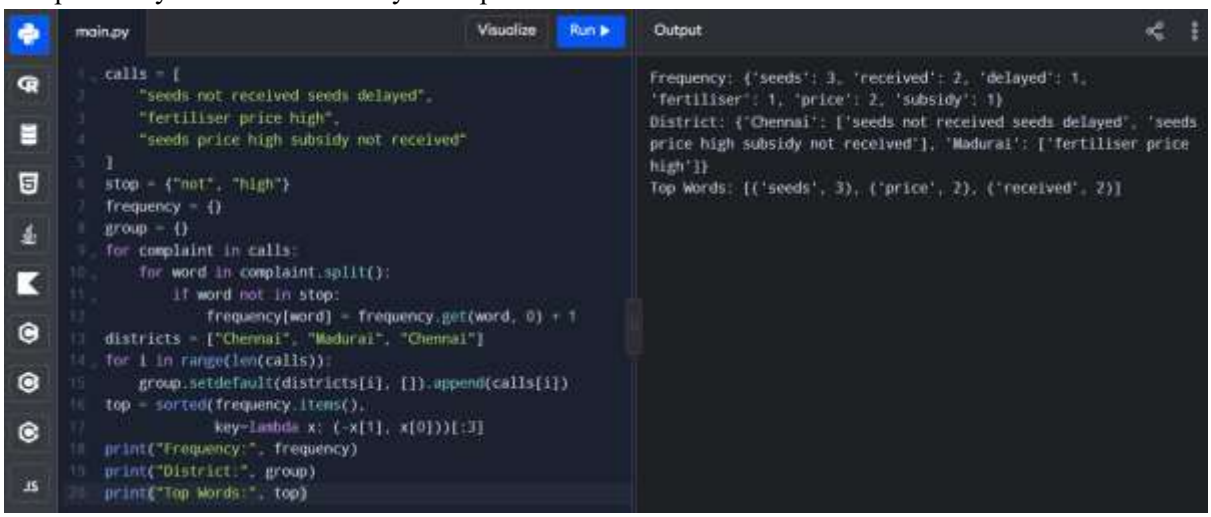
Find top three words

Display results

STOP

Result:

Thus, the Python program was executed successfully to count complaint frequencies, group complaints by district and identify the top three words.



Q4 – Sets: Elective Registration Clash Detection

Aim:

To write a Python program using sets to identify students registered in both electives, exactly one elective and neither elective.

Algorithm:

1. Start the program.
2. Create two elective sets.
3. Create a set of all students.
4. Find common students using intersection.
5. Find exactly one elective using symmetric difference.
6. Find students in neither elective using difference.
7. Sort and display the results.
8. Stop the program.

Pseudocode:

START

Create AI elective set

Create IoT elective set

Create all students set

Find intersection

Find symmetric difference

Find students in neither elective

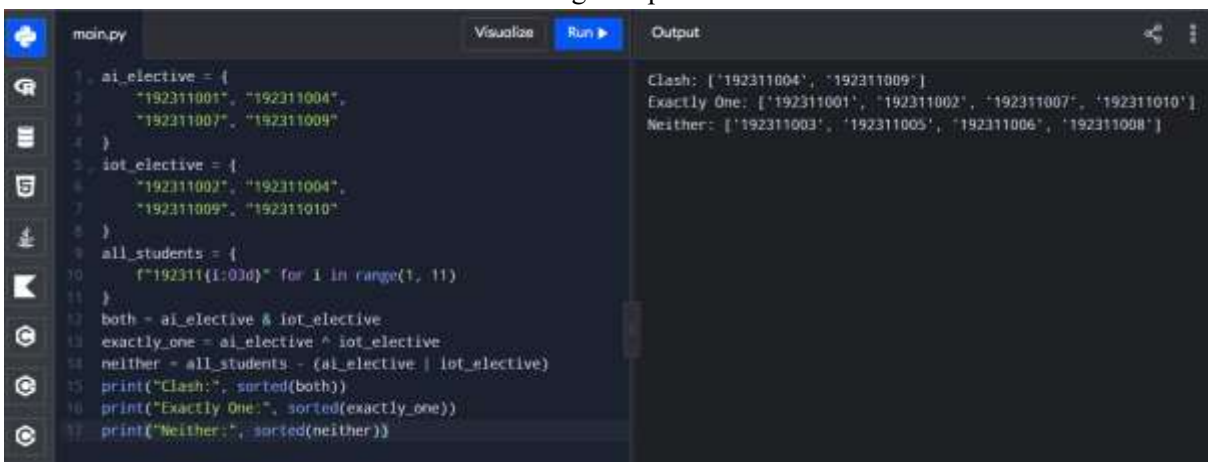
Sort results

Display results

STOP

Result:

Thus, the Python program was executed successfully to identify elective clashes, students in exactly one elective and students in neither elective using set operations.



Q5 – List Comprehension: Cleaning Sensor Readings

Aim:

To write a Python program using list comprehension, map(), filter() and enumerate() to clean sensor readings and identify alarm minutes.

Algorithm:

1. Start the program.
2. Create a list of temperature readings.
3. Remove invalid readings.
4. Convert Celsius into Fahrenheit using map().
5. Filter alarm readings using filter().
6. Identify alarm minutes using enumerate().
7. Display the results.
8. Stop the program.

Pseudocode:

START

Create temperature list

Create empty valid list

FOR each reading

IF reading is between -50 and 50

Add reading to valid list

END IF

END FOR

Convert Celsius to Fahrenheit

Filter alarm readings

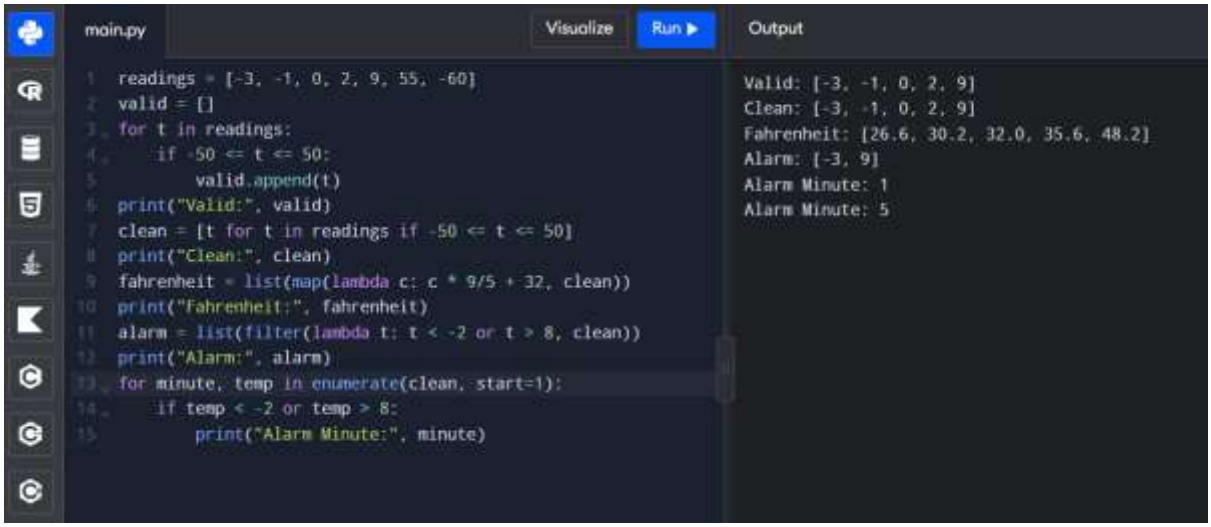
Find alarm minutes

Display results

STOP

Result:

Thus, the Python program was executed successfully to clean sensor readings, convert temperatures and identify alarm minutes.



Q6 – Nested Data: Railway Timetable

Aim:

To write a Python program using nested dictionaries, lists and tuples to store railway timetable information and find trains stopping at a particular station on a given day.

Algorithm:

1. Start the program.
2. Create a nested dictionary for trains.
3. Store train names, station details and running days.
4. Enter the required station and day.
5. Search for matching trains.
6. Check station and running day.
7. Sort trains by departure time.
8. Display the timetable.
9. Stop the program.

Pseudocode:

START

Create railway timetable

Input station and day

Create empty result list

FOR each train

    Check running day

    FOR each station

        IF station matches

            Add train details

        END IF

    END FOR

END FOR

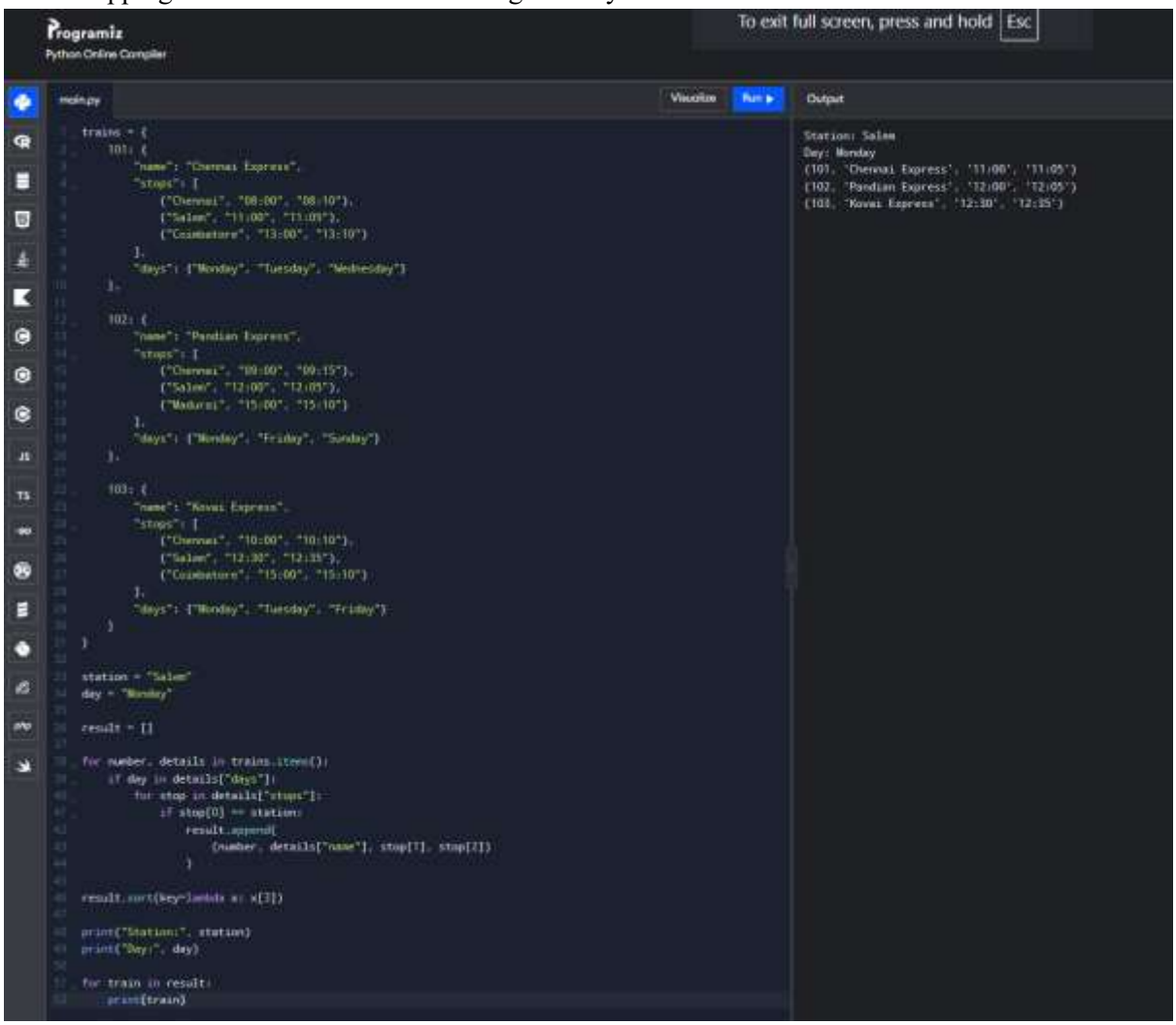
Sort by departure time

Display timetable

STOP

Result:

Thus, the Python program was executed successfully to store railway timetable details and display trains stopping at the selected station on the given day.



Q7 – Data Structure Design: Contact-Merging Program

Aim:

To write a Python program using dictionaries and sets to merge duplicate phone contacts by normalising phone numbers and names and combining email addresses.

Algorithm:

1. Start the program.
2. Create a list of contacts.
3. Normalise phone numbers by removing spaces and symbols.
4. Convert names into lowercase.
5. Create an empty dictionary.
6. Check whether the phone number already exists.
7. Merge duplicate contact details using sets.

8. Display merged contacts.
9. Stop the program.

Pseudocode:

```
START
Create contact list
Create empty dictionary
FOR each contact
    Normalise phone number
    Normalise name
    IF phone number exists
        Merge contact details
    ELSE
        Create new contact
    END IF
    Add phone number and email
END FOR
Display merged contacts
STOP
```

Result:

Thus, the Python program was executed successfully to normalise contact details and merge duplicate phone contacts using dictionaries and sets.

